

Relational Algebra

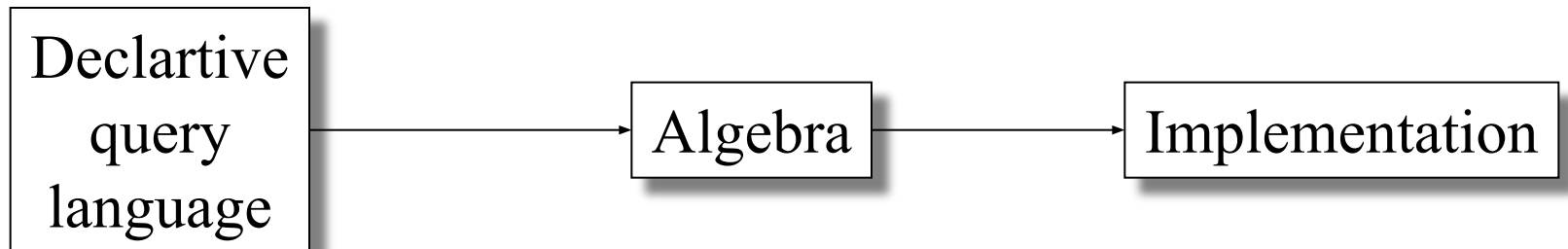
DBMS Architecture

How does a SQL engine work ?

- SQL query $\rightarrow$ relational algebra plan
- Relational algebra plan $\rightarrow$ Optimized plan
- Execute each operator of the plan

Relational Algebra

- Formalism for creating new relations from existing ones
- Its place in the big picture:



SQL,
relational calculus

Relational algebra
Relational bag algebra

Relational Algebra

- Five operators:
 - Union: $\cup$
 - Difference: $-$
 - Selection: σ
 - Projection: Π
 - Cartesian Product: $\times$
- Derived or auxiliary operators:
 - Intersection, complement
 - Joins (natural, equi-join, theta join, semi-join)
 - Renaming: ρ

1. Union and 2. Difference

- $R1 \cup R2$
- Example:
 - $\text{ActiveEmployees} \cup \text{RetiredEmployees}$
- $R1 - R2$
- Example:
 - $\text{AllEmployees} - \text{RetiredEmployees}$

What about Intersection ?

- It is a derived operator
- $R1 \cap R2 = R1 - (R1 - R2)$
- Also expressed as a join (will see later)
- Example
 - `UnionizedEmployees` $\cap$ `RetiredEmployees`

3. Selection

- Returns all tuples which satisfy a condition
- Notation: $\sigma_c(R)$
- Examples
 - $\sigma_{\text{Salary} > 40000}(\text{Employee})$
 - $\sigma_{\text{name} = \text{"Smith"}}(\text{Employee})$
- The condition c can be $=, <, \leq, >, \geq, \neq$

SSN	Name	Salary
1234545	John	200000
5423341	Smith	600000
4352342	Fred	500000

$\sigma_{\text{Salary} > 40000}$ (Employee)

SSN	Name	Salary
5423341	Smith	600000
4352342	Fred	500000

4. Projection

- Eliminates columns, then removes duplicates
- Notation: $\Pi_{A_1, \dots, A_n}(R)$
- Example: project social-security number and names:
 - $\Pi_{SSN, Name}(Employee)$
 - Output schema: Answer(SSN, Name)

SSN	Name	Salary
1234545	John	200000
5423341	John	600000
4352342	John	200000

$\Pi_{\text{Name,Salary}}$ (Employee)

Name	Salary
John	20000
John	60000

5. Cartesian Product

- Each tuple in R1 with each tuple in R2
- Notation: $R1 \times R2$
- Example:
 - Employee $\times$ Dependents
- Very rare in practice; mainly used to express joins

Cartesian Product Example

Employee

Name	SSN
John	999999999
Tony	777777777

Dependents

EmployeeSSN	Dname
999999999	Emily
777777777	Joe

Employee x Dependents

Name	SSN	EmployeeSSN	Dname
John	999999999	999999999	Emily
John	999999999	777777777	Joe
Tony	777777777	999999999	Emily
Tony	777777777	777777777	Joe

Relational Algebra

- Five operators:
 - Union: $\cup$
 - Difference: $-$
 - Selection: σ
 - Projection: Π
 - Cartesian Product: $\times$
- Derived or auxiliary operators:
 - Intersection, complement
 - Joins (natural, equi-join, theta join, semi-join)
 - Renaming: ρ

Renaming

- Changes the schema, not the instance
- Notation: $\rho_{B_1, \dots, B_n}(R)$
- Example:
 - $\rho_{\text{LastName}, \text{SocSocNo}}(\text{Employee})$
 - Output schema:
Answer(LastName, SocSocNo)

Renaming Example

Employee

Nam	SS
John	00 99999999
Tony	777777777

$\rho_{\text{LastName, SocSocNo}}$ (**Employee**)

LastNam	SocSocNo
John	999999999
Tony	777777777

Natural Join

- Notation: $R1 \bowtie R2$
- Meaning: $R1 \bowtie R2 = \Pi_A(\sigma_C(R1 \times R2))$
- Where:
 - The selection σ_C checks equality of all common attributes
 - The projection eliminates the duplicate common attributes

Natural Join

Example

Employee	
Nam	SS
John	0999999999
Tony	7777777777

Dependent	
SS	Dnam
0999999999	Emil
7777777777	Joe

Employee $\bowtie$ **Dependents** =

$\Pi_{\text{Name, SSN, Dname}}(\sigma_{\text{SSN}=\text{SSN2}}(\text{Employee} \times \rho_{\text{SSN2, Dname}}(\text{Dependents})))$

Nam	SS	Dnam
John	0999999999	Emil
Tony	7777777777	Joe

Theta Join

- A join that involves a predicate
- $R1 \bowtie_{\theta} R2 = \sigma_{\theta}(R1 \times R2)$
- Here θ can be any condition

Eq-join

- A theta join where θ is an equality
- $R1 \bowtie_{A=B} R2 = \sigma_{A=B} (R1 \times R2)$
- Example:
 - $\text{Employee} \bowtie_{\text{SSN}=\text{SSN}} \text{Dependents}$
- Most useful join in practice